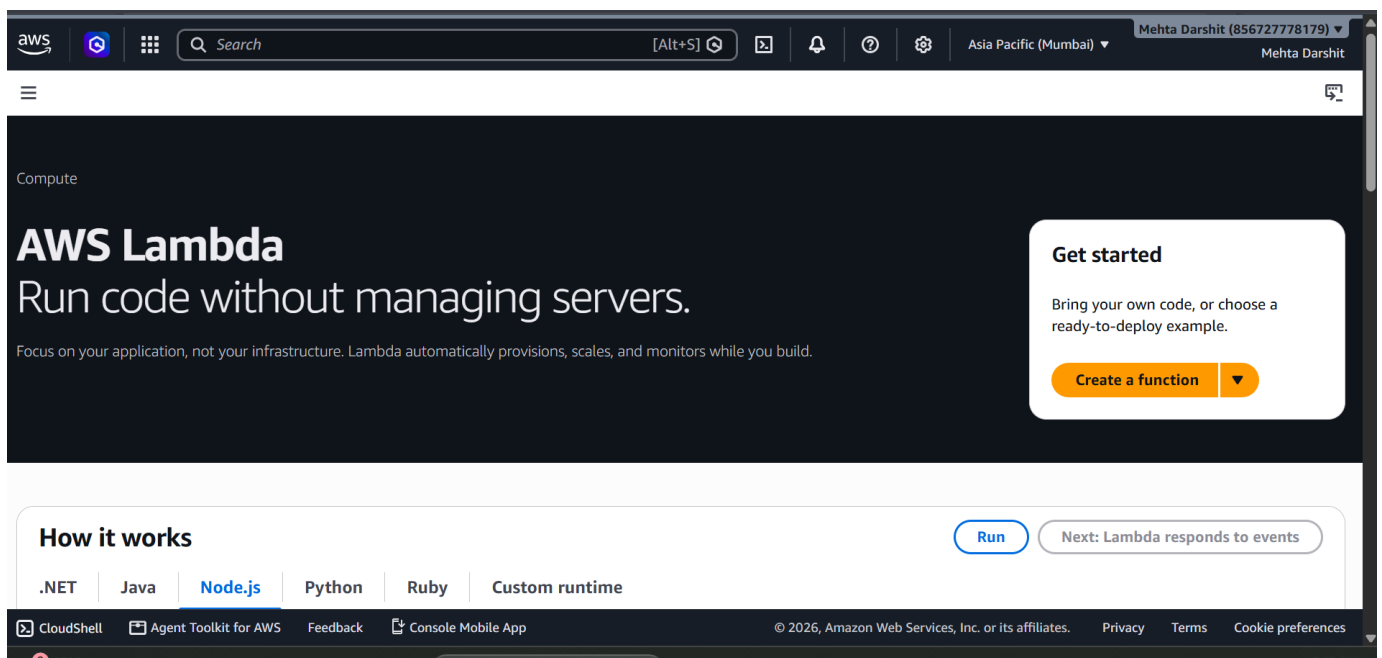


Introduction to serverless computing

Task-1. Sign up for a free AWS account (if you don't have one), log in to the AWS Management Console, and navigate to the AWS Lambda service dashboard.

- ? Go to [AWS](#).
- ? Select **Create an AWS Account** if you don't already have one.
- ? Complete the AWS signup and verification steps.
- ? Sign in to the **AWS Management Console**.
- ? In the console search bar, type **Lambda** and open **AWS Lambda**.
- ? You should now be on the Lambda dashboard.



Task-2. Create a new AWS Lambda function named InstaLikeCounter that takes an event with a 'likes' number and returns the value increased by 1, simulating an Instagram-like button click.

In AWS Lambda:

1. Click **Create function**.
2. Choose **Author from scratch**.
3. Set:
 - o **Function name:** InstaLikeCounter

- **Runtime:** Python 3.x
- 4. Leave the default execution role settings unless your course specifies otherwise.
- 5. Click **Create function**.
- 6. In the code editor, replace the generated code with:

```
def lambda_handler(event, context):
```

```
    print("Function executed!")
```

```
    likes = event.get("likes", 0)
```

```
    return {
```

```
        "likes": likes + 1
```

```
    }
```

Click **Deploy**.

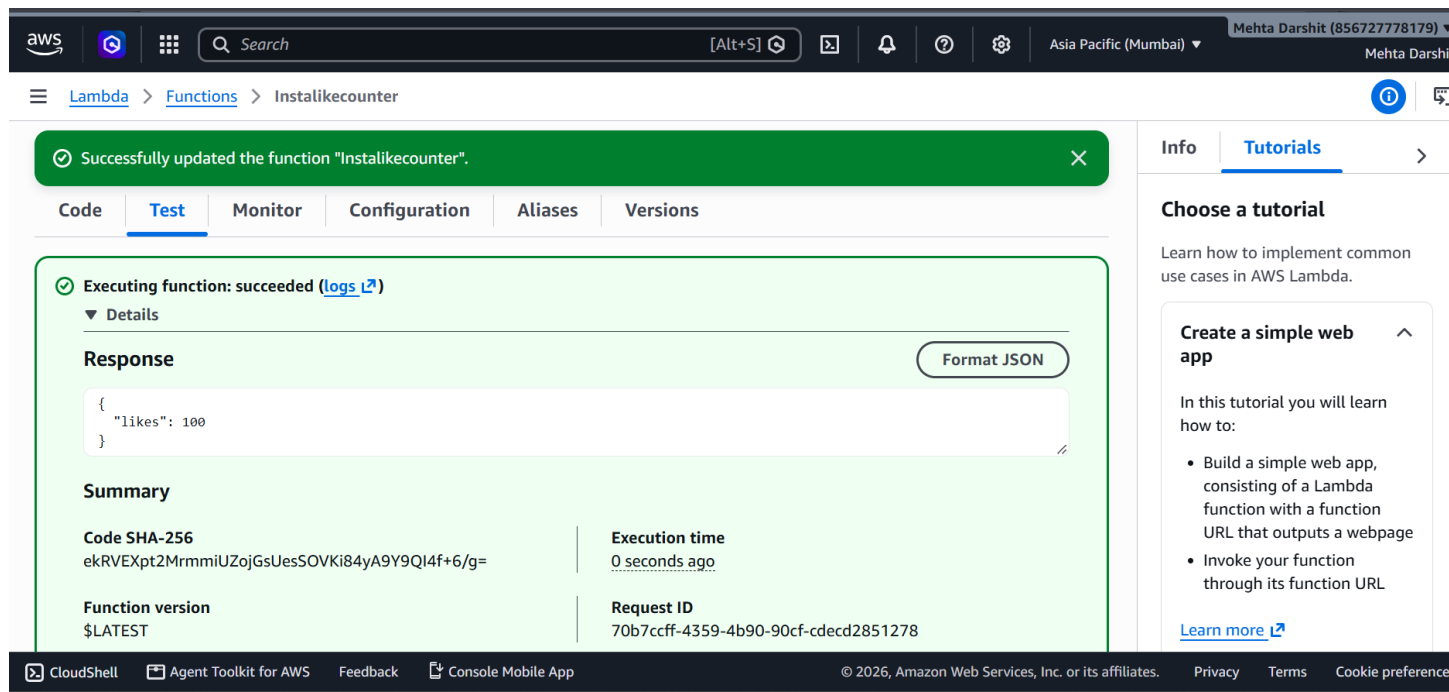
The screenshot displays the AWS Lambda console interface. At the top, the navigation bar shows the AWS logo, a search bar, and the region 'Asia Pacific (Mumbai)'. The breadcrumb trail indicates the path: Lambda > Functions > Instalikecounter. The main content area is titled 'Code source' and features a file explorer on the left showing the 'INSTALIKECOUNTER' folder with a file named 'lambda_function.py'. The central editor displays the Python code for the lambda handler, which prints 'Function executed!' and increments the 'likes' count. On the right side of the editor, there are buttons for 'Open in Visual Studio Code' and 'Update'. Below the code editor, there are 'Deploy (Ctrl+Shift+U)' and 'Test (Ctrl+Shift+I)' buttons. The right sidebar contains a 'Tutorials' section with a link to 'Create a simple web app'. The bottom status bar shows the system clock, weather (32°C Sunny), and various application icons.

Task-3. Test your InstaLikeCounter Lambda function using the built-in AWS Lambda test feature by providing a sample event with 'likes': 99, and verify the output shows 100.

Create a test event using:

```
{  
  
  "likes": 99  
}
```

Then click **Test**.



The screenshot displays the AWS Lambda console interface. At the top, a green notification bar states "Successfully updated the function 'Instalikecounter'." Below this, the "Test" tab is selected, showing the execution details for the "Instalikecounter" function. The response is a JSON object: {"likes": 100}. The summary section provides additional details: Code SHA-256 (ekRVEXpt2MrmmlUzJGsUesSOVKi84yA9Y9QI4f+6/g=), Execution time (0 seconds ago), Function version (\$LATEST), and Request ID (70b7ccff-4359-4b90-90cf-cdec2851278). The right sidebar contains a "Choose a tutorial" section with a link to "Create a simple web app".

Task-4. Write a short comparison (3-4 lines each) of AWS Lambda, Azure Functions, and Google Cloud Functions, focusing on how each lets you run code without managing servers.

AWS Lambda

Runs code in response to events without requiring you to provision or maintain servers.

It automatically scales based on incoming requests.

You pay based largely on invocations and compute duration.

Azure Functions

Microsoft's serverless compute service for running event-driven code.

It handles infrastructure and scaling automatically.

It integrates particularly well with Azure services and Microsoft development tools.

Google Cloud Functions

Google's serverless platform for executing small pieces of code triggered by events or HTTP requests.

Google manages the underlying servers and automatically scales the function.

It integrates closely with Google Cloud services such as Cloud Storage and Pub/Sub.

Task-5. Use the AWS Lambda code editor to add a `console.log` statement that prints 'Function executed!' every time your function runs. Check the Monitor tab after running your test to see the log output.

After running your test:

1. Open the **Monitor** tab for InstaLikeCounter.
2. Select **View CloudWatch logs / View logs in CloudWatch**.
3. Open the latest log stream.
4. Look for:

Function executed!

The screenshot shows the AWS CloudWatch console interface. The top navigation bar includes the AWS logo, a search bar, and the user's name 'Mehta Darshit'. The breadcrumb trail indicates the path: CloudWatch > Log management > /aws/lambda/Instalikecounter > 2026/10/06/[LATEST]677df5215f8240039183e29f819030cd. The left sidebar shows the 'CloudWatch' menu with options like Ingestion, Dashboards, Alarms, AI Operations, GenAI Observability, Application Signals (APM), Infrastructure Monitoring, and Logs. The main content area is titled 'Log events' and contains a filter bar with a search input and a 'Display' button. Below the filter bar, a table of log events is displayed. The events show the function's execution, including the 'Function executed!' message. The bottom of the console features a footer with links to CloudShell, Agent Toolkit for AWS, Feedback, Console Mobile App, and copyright information for Amazon Web Services.

Timestamp	Message
2026-10-06T06:50:18.467Z	INIT_START Runtime Version: python:3.14.mainline.v75 Runtime Version ARN: arn:aws:la
2026-10-06T06:50:18.570Z	START RequestId: 70b7ccff-4359-4b90-90cf-cdec2851278 Version: \$LATEST
2026-10-06T06:50:18.571Z	Function executed!
2026-10-06T06:50:18.573Z	END RequestId: 70b7ccff-4359-4b90-90cf-cdec2851278
2026-10-06T06:50:18.573Z	REPORT RequestId: 70b7ccff-4359-4b90-90cf-cdec2851278 Duration: 1.76 ms Billed Dura